Images, media, and backgrounds

In the **HTML pack**, you learned that images and media need meaningful HTML: `src`, `alt`, captions, controls, and useful fallback text. CSS has a different job for that media.

CSS decides how that media fits into the page: how wide it is, whether it keeps its shape, whether it crops, and whether an image is real content or just a decorative background.

Start with a media card

Before you start, save a landscape photo, roughly `1200px` wide and `800px` tall, with the name `course-cover.jpg` in the same folder as your HTML. You can **download this sample image** if you need one. If you choose a different filename, make sure to update the `src` and `background-image` values in the code below.

Use two files for this media exercise. Start with this `index.html`:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Image sizing practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Images, media, and backgrounds</h1>
        <p class="summary">Media has its own natural size. CSS helps it fit the boxes or
      </section>

      <article class="course-card">
        <img class="course-image" src="course-cover.jpg" alt="Describe your practice ima

        <div class="course-body">
          <p class="badge">Featured lesson</p>
          <h2>Design systems for beginners</h2>
          <p class="course-text">Learn how spacing, color, and reusable components make
        </div>
      </article>

      <section class="promo">
        <p class="eyebrow">Workshop</p>
        <h2>Build a visual landing page</h2>
        <p>Use one strong image, readable text, and simple spacing.</p>
      </section>
    </main>
```

```
  </body>
</html>
```

Then add the starting CSS:

css

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.course-card,
.promo {
  margin-bottom: 24px;
}

.intro,
.course-card {
  background-color: white;
  border: 1px solid #e5e7eb;
```

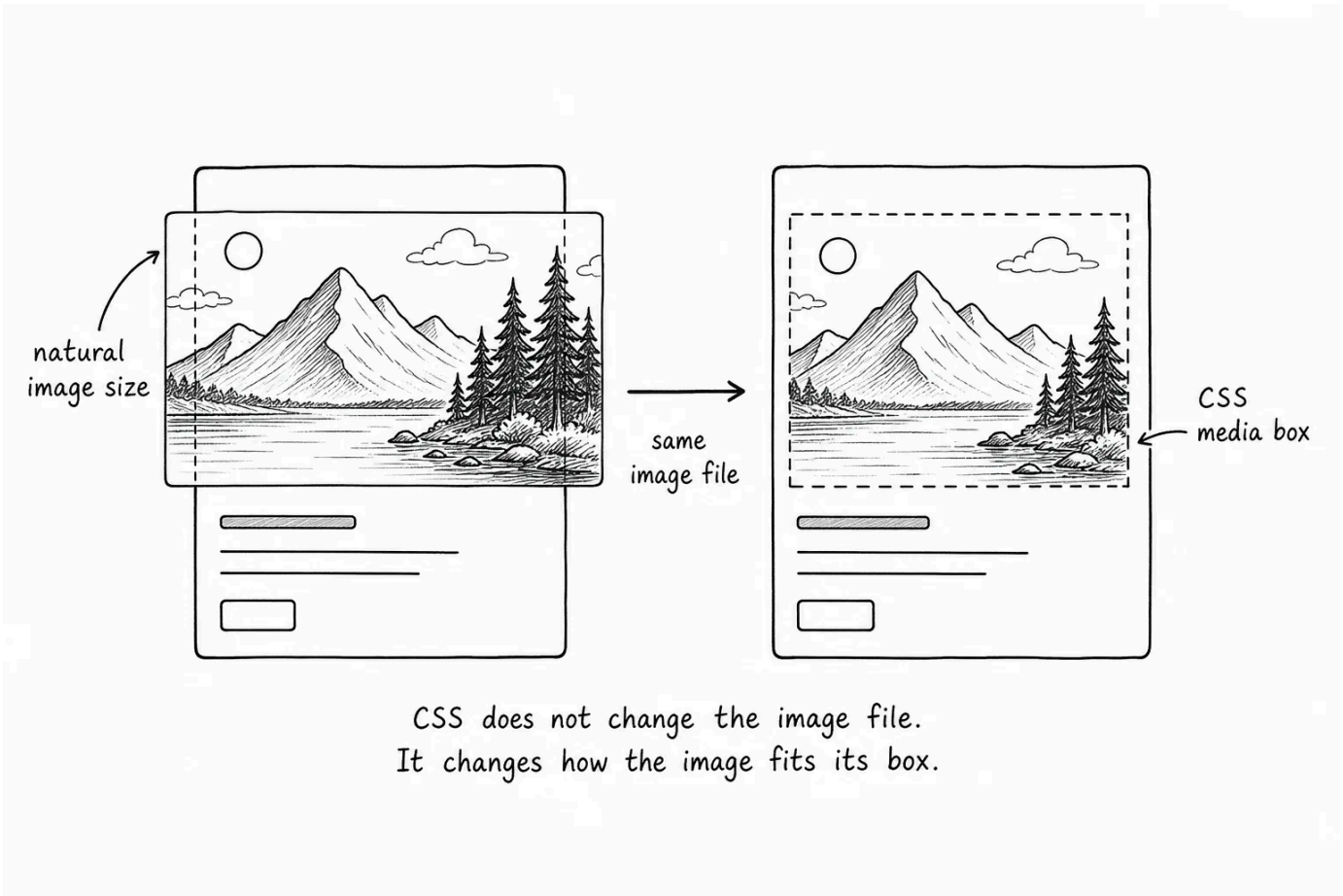
Previous Lesson Next Lesson

```
.intro,
.course-body,
.promo {
  padding: 24px;
}

.eyebrow {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.course-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  display: inline-block;
  color: #1d4ed8;
  background-color: #eff6ff;
  padding: 6px 10px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
}

.course-image {
  border-radius: 8px 8px 0 0;
}

.promo {
  color: white;
  background-color: #111827;
  border-radius: 8px;
}

.promo .eyebrow {
  color: #bfdbfe;
```

```
}
```

Open the page and you will notice that the image may look too large and awkward inside the card. That is not a failure. It is the image showing at its natural size.

CSS needs to tell the image how to fit the card.



Images have a natural size

An image file has its own dimensions. A photo might be `1600px` wide and `1000px` tall. An icon might be `64px` wide and `64px` tall. If you do not give the browser any CSS sizing instructions, the image uses its natural size.

The usual first fix is to give the image a fluid width and let its height follow naturally:

```css
.course-image {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px 8px 0 0;
}
```

Reload the page and the image should now fit inside the card.

`width: 100%` makes the image fill the width of the card.

`height: auto` preserves the image's natural aspect ratio, so the image does not stretch.

`display: block` connects back to the last lesson. Images are inline by default, which can leave a tiny bit of space under them. Making the image

block-level removes that inline spacing behavior and makes the card edge cleaner.

> **Start with fluid/responsive images** - For normal content images, `display: block; width: 100%; height: auto;` is a safe beginner pattern. It makes the image fit its container without distorting it.

Crop an image with object-fit

Sometimes you want every card image to have the same shape, even if the original images have different dimensions. You can do that with `aspect-ratio` and `object-fit`. For the sample image, update the style like this:

```css
.course-image {
  display: block;
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover;
  object-position: center;
  border-radius: 8px 8px 0 0;
}
```

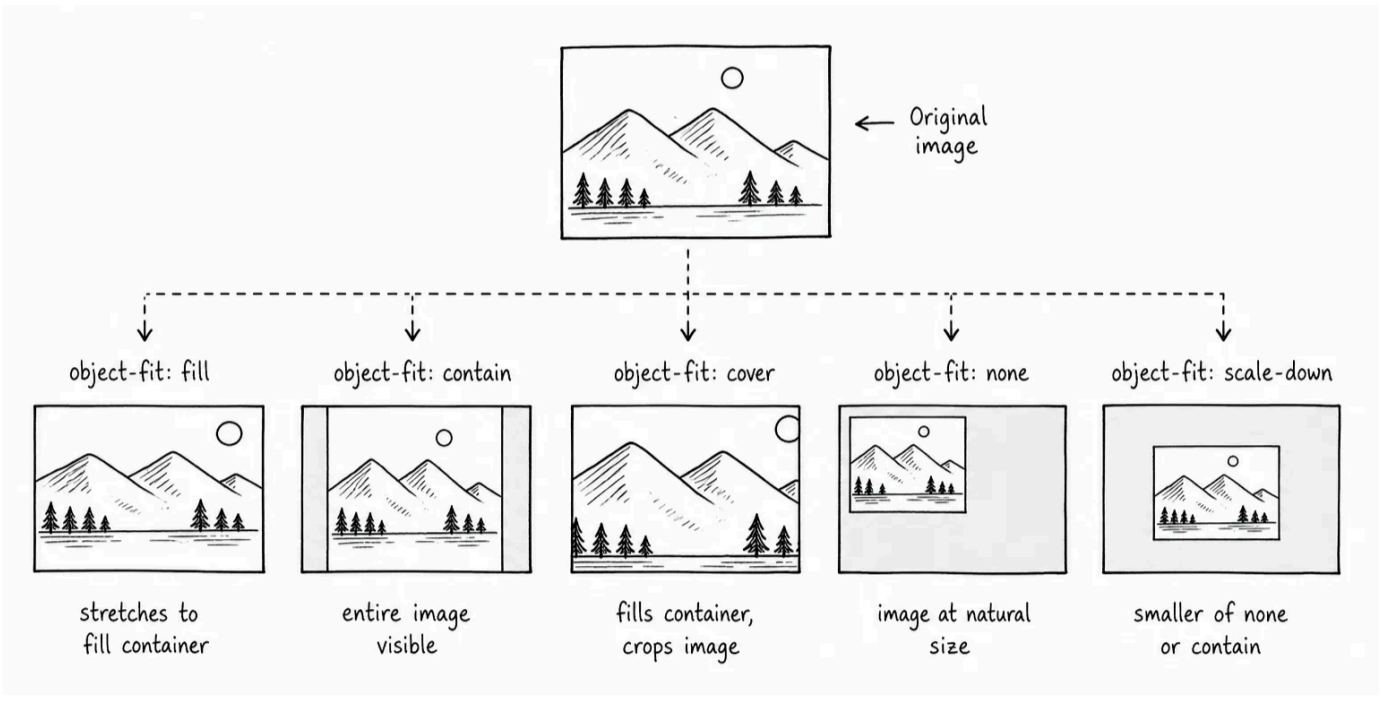
Reload the page and notice how the image now has a consistent wide shape.

`aspect-ratio: 16 / 9` gives the image box a consistent wide shape.

AI Tutor What is aspect ratio for images in CSS?

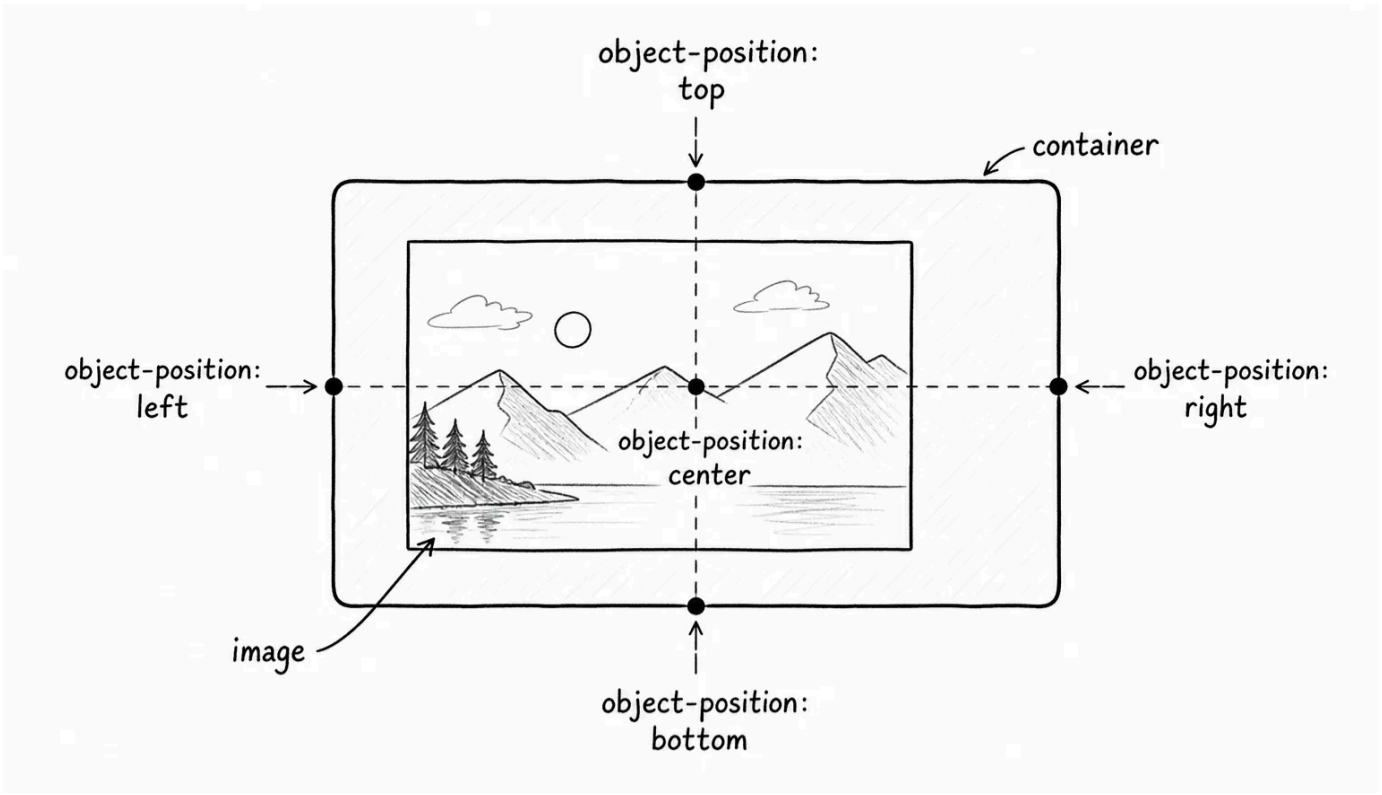
`object-fit: cover` fills that box without stretching the image. If the image does not match the box shape, the browser crops the extra part.

There are other possible values for `object-fit` as well. The image below shows how each behaves for an image inside a container.



`object-position: center` chooses which part of the image stays centered during the crop applied by `object-fit`. The image below shows different

`object-position` values and their impact.



Try changing `object-position` values and see how they affect the crop.

Avoid stretching images

If you assign a fluid width and set a fixed height, it usually results in an awkward stretched image:

```css
.course-image {
  width: 100%;
  height: 240px;
}
```

Without `object-fit`, the browser stretches the image to fit both dimensions. Faces, logos, and screenshots can look squeezed.

If you need to set a fixed height for an image box, you usually add `object-fit` too:

```css
.course-image {
  width: 100%;
  height: 240px;
  object-fit: cover;
}
```

That tells the browser to crop instead of distort.

> ⚠ **Check for stretched images** - If an image looks stretched, check whether both `width` and `height` are being forced without `object-fit`. Cropping is usually better than distortion, but important content can still be cropped away, so inspect the result.

Use background images for decoration

The card image is real content. It is in the HTML as `<img>`, and it has `alt` text.

The HTML also has a `.promo` section near the end. Use the same image as a decorative background with `background-image`:

```css
.promo {
  color: white;
  background-color: #111827;
  background-image: url("course-cover.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  border-radius: 8px;
  padding: 32px;
}
```

Notice that background images use `background-size` and `background-position` instead of `object-fit` and `object-position`. Reload the page and you should see the image filling the promo box.

Depending on your image, the text may be hard to read. You can fix that by adding a dark overlay to the background image:

```css
.promo {
  color: white;
  background-color: #111827;
  background-image:
    linear-gradient(rgba(17, 24, 39, 0.78), rgba(17, 24, 39, 0.78)),
    url("course-cover.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  border-radius: 8px;
  padding: 32px;
}
```

Reload the page and you should see a slightly darkened background image.

`background-image` paints an image behind the content.

`linear-gradient(fromColor, toColor)` creates a gradient. In this example, both color stops are the same dark transparent color, so it acts like a solid overlay. CSS gradients count as images, which is why they can sit inside `background-image` with `url(...)`.

`background-size: cover` makes the background fill the section.

`background-position: center` chooses which part of the image is centered.

`background-repeat: no-repeat` prevents the image from tiling if the background does not fully cover the box.

The overlay darkens the image so the text stays readable. You used `rgba()` earlier for transparent colors. This is the same idea applied as a background layer.

> ⚠ **Background images do not have alt text** - A CSS background image is not content. Screen readers do not get `alt` text for it. If the image is important to understanding the page, use `<img>` in the HTML instead.

Choose img or background-image

Use an `<img>` when the image is content:

- Product photo.
- Avatar.
- Chart or diagram.
- Screenshot.
- Article image that helps explain the article.

Use `background-image` when the image is decoration:

- Texture.
- Abstract hero background.
- Decorative pattern.
- Image behind text where the text already says the important thing.

The difference is not only visual. It affects accessibility, loading behavior, search engines, and how easy the image is to replace from app data.

You just saw the accessibility angle: background images have no `alt` text. The same choice matters for search engines, which can understand meaningful `<img>` content more easily than decorative backgrounds. It also matters in real apps, where an image from product data, user data, or article data usually belongs in HTML, not hidden inside a CSS file.

Choose meaning first, then style the image to fit that meaning.

Media elements are boxes too

In the **HTML lesson pack**, you learned about videos and embedded media, and those media elements need sizing too.

You do not need to add this video example to your practice page. It is here so you can recognize the pattern later when you embed videos or third-party players.

For a normal video element, the same responsive idea works:

```css
video {
  display: block;
  width: 100%;
  height: auto;
}
```

For an embedded video inside an iframe, developers often wrap the iframe in a box with an aspect ratio:

```html
<div class="video-frame">
  <iframe src="https://www.youtube.com/embed/example" title="Course preview"></iframe>
</div>
```

```css
.video-frame {
  aspect-ratio: 16 / 9;
}

.video-frame iframe {
  display: block;
  width: 100%;
  height: 100%;
  border: 0;
}
```

You do not need to memorize every media pattern yet.

The mental model is enough: media has a shape, and CSS controls how that shape fits the box.

Try it

Before moving on, test how the same image behaves under different fitting rules:

- Remove `width: 100%;` from `.course-image` and reload. Notice the image return toward its natural size.
- Add `height: 240px;` to `.course-image` and temporarily remove `object-fit: cover;`. Notice whether the image stretches.

- Change `aspect-ratio: 16 / 9;` to `aspect-ratio: 1 / 1;`. Notice the image box become square.
- Change `object-fit: cover;` to `object-fit: contain;`. Notice that the whole image appears, but the box may show empty space.
- Change `object-position: center;` to `object-position: top;`.
- Remove the `linear-gradient(...)` layer from `.promo`. Notice whether the text is still readable on top of the image.
- Change `background-size: cover;` to `background-size: contain;`. Notice how the background behaves differently from the `<img>`.
- Remove `background-repeat: no-repeat;` from `.promo`. Notice whether the background image repeats.

The goal is to see images as boxes with fitting rules. When an image looks wrong, ask whether the problem is size, aspect ratio, cropping, or whether the image should be content instead of decoration.

Next Lesson →